

1) Compare between manual heap and stl priority queue:

1. Manual heap :

- **Advantages:**

1. Code logic full control (high learning value)
2. Can modify behavior easily (like severityupdate)

- **Disadvantages:**

1. Longer and more complex code
2. High chance of bugs
3. Manually maintaining heap property

- **Time Complexity:**

1. heapify_up : $O(\log n)$
2. heapify_down : $O(\log n)$
3. insert : $O(\log n)$
4. treat_next_patient : $O(\log n)$
5. view_next_patient : $O(1)$
6. update_severity : $O(n)$
7. display_all_patients : $O(n)$
8. swap : $O(1)$

- **Ease of use:**

The manual heap is more complex to implement and use because it requires a manual writing and maintaining heap operations and requires a deeper understanding of how heaps work but it provides full control over the logic

2. STL priority queue:

- **Advantages:**

1. Short code
2. Easy to use
3. Already optimized and tested
4. Less chance of bugs

- **Disadvantages:**

1. Limited flexibility
2. Hard to update
3. Hidden logic (less learning value)

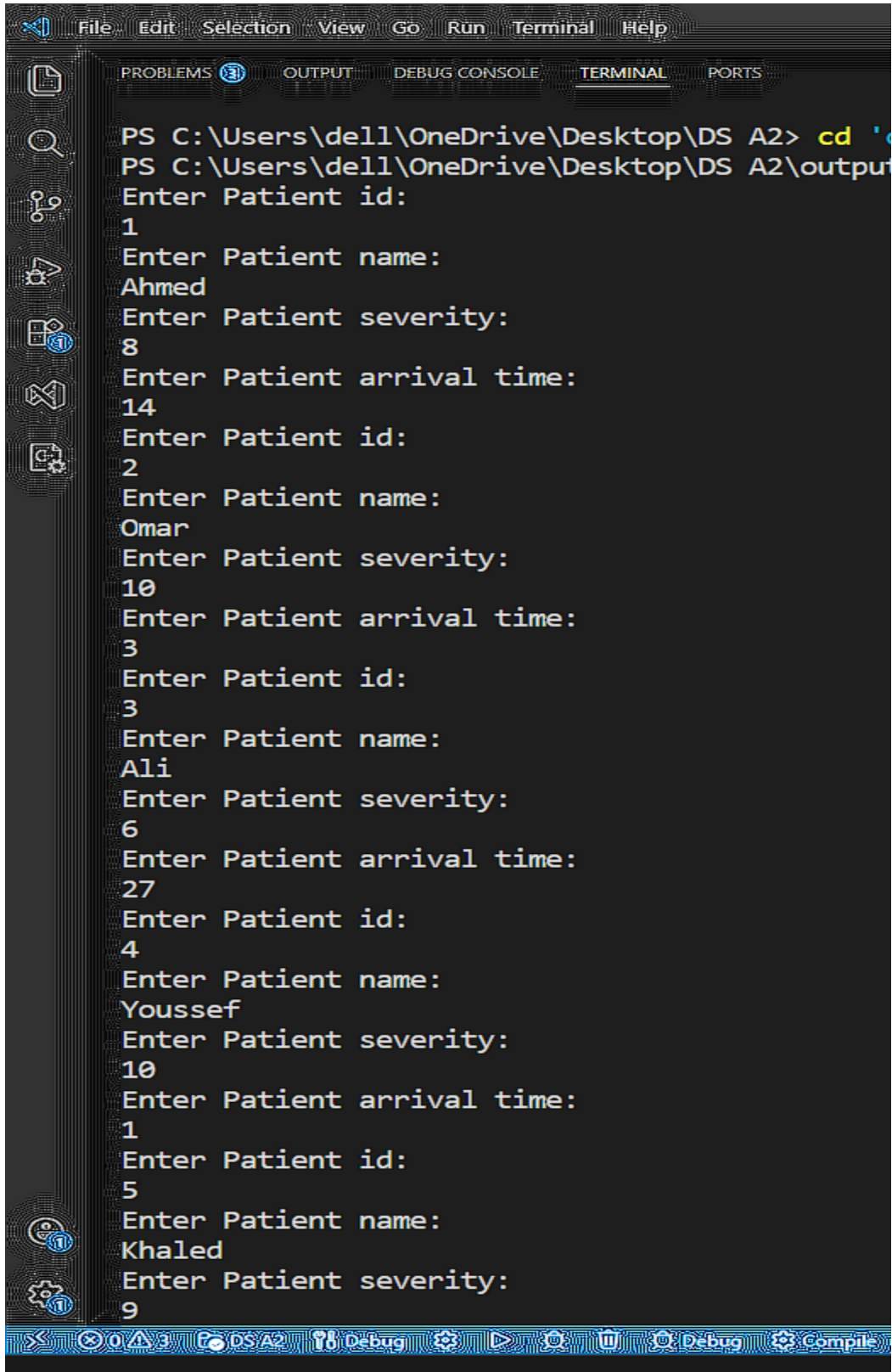
- **Time Complexity:**

1. push : $O(\log n)$
2. pop : $O(\log n)$
3. top : $O(1)$
4. update_severity : $O(n \log n)$ (rebuild queue)
5. display_all_patients : $O(n \log n)$ (copy+pop)

- **Ease of Use:**

The STL priority queue is very easy to use because it requires minimal code and faster to develop but it provides less flexibility, especially when updating, which is not directly supported.

Sample run:



```
PS C:\Users\dell\OneDrive\Desktop\DS A2> cd 'C:\Users\dell\OneDrive\Desktop\DS A2\output'
Enter Patient id:
1
Enter Patient name:
Ahmed
Enter Patient severity:
8
Enter Patient arrival time:
14
Enter Patient id:
2
Enter Patient name:
Omar
Enter Patient severity:
10
Enter Patient arrival time:
3
Enter Patient id:
3
Enter Patient name:
Ali
Enter Patient severity:
6
Enter Patient arrival time:
27
Enter Patient id:
4
Enter Patient name:
Youssef
Enter Patient severity:
10
Enter Patient arrival time:
1
Enter Patient id:
5
Enter Patient name:
Khaled
Enter Patient severity:
9
```

```
File Edit Selection View Go Run Terminal Help
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

1- View next patient
2- Treat next patient
3- Update severity
4- Display all patients
5- Insert new patient
6- Exit
1
ID: 4
Name: Youssef
Severity: 10
Arrival Time: 1
-----
1- View next patient
2- Treat next patient
3- Update severity
4- Display all patients
5- Insert new patient
6- Exit
5
Enter Patient id:
7
Enter Patient name:
mo
Enter Patient severity:
10
Enter Patient arrival time:
0
1- View next patient
2- Treat next patient
3- Update severity
4- Display all patients
5- Insert new patient
6- Exit
1
ID: 7
Name: mo
Severity: 10
Arrival Time: 0
```

Report:

1)How heap works:

A heap is a special type of binary tree used to store data in a way that makes it easy to quickly access the highest or lowest priority element.

In the project we used a max heap, where:

- The patient with the highest severity is at the top.
- If two patients have the same severity, the one who arrives first treated first.

The heap is stored in a vector, and it is maintained using two main operations:

- heapify up: used when inserting a new patient to move it to the correct position.
- heapify down: used when removing a patient to restore the heap structure.

2)Why it's efficient:

The heap is efficient because it keeps the most important element (highest priority) at the top.

This allows:

- Fast access to the next patient : $O(1)$
- Fast insertion : $O(\log n)$
- Fast removal : $O(\log n)$

Instead of searching through all patients, the heap organizes them so that operations are much faster.

3)Comparison with other structures:

Manual Heap:

1)Characteristics

- Stores data in heap order
- Highest priority element is always at the top
- Implemented using a vector
- Uses heapify operations to maintain order

2)Time Complexity

- Insert : $O(\log n)$
- Delete highest priority : $O(\log n)$
- View highest priority : $O(1)$

Best Use

- hospital emergency rooms
- task scheduling
- CPU scheduling

Array / Vector:

1)Characteristics

- Elements stored sequentially
- No automatic ordering

2)Time Complexity

- Insert : $O(1)$
- Find highest priority : $O(n)$
- Delete highest priority : $O(n)$

Comparison with Heap

Arrays are simpler, but heaps are much faster for priority operations because arrays require searching through all elements.

Linked List:

1)Characteristics

- Elements connected using pointers
- Dynamic memory allocation

2)Time Complexity

- Insert : $O(1)$
- Find highest priority : $O(n)$
- Delete highest priority : $O(n)$

Comparison with Heap

Linked lists allow easy insertion, but finding the highest priority patient is slow because all nodes must be checked.

Sorted Array:

1)Characteristics

- Elements always kept sorted

2)Time Complexity

- Insert : $O(n)$
- View highest priority : $O(1)$
- Delete highest priority : $O(1)$

Comparison with Heap

Sorted arrays provide fast access to the top element, but insertion is slow because elements must be shifted.

Binary Search Tree (BST):

1)Characteristics

- Tree structure with ordered nodes

2)Time Complexity

- Average insert/search/delete : $O(\log n)$
- Worst case : $O(n)$

Comparison with Heap

BST supports searching better than heaps, but heaps are better for repeatedly accessing the highest priority element.